CODE.TALKS 2026 · HAMBURG · 5.11.

Architecture Overview gegen Comprehension Debt

Ein Erfahrungsbericht aus meinem Labor

Nils Eckelt · ecke.lt

WER HIER SPRICHT

Nils Eckelt

Interims-CTO und Trainer für Agentic Coding aus Hamburg

- Code schreibe ich, seit ich 15 bin
- Software Engineer, UX Designer, Agile Coach, DevOps, Cloud Architect, Entwicklungsleiter
- Seit 2025 bringe ich Agentic Coding in Teams und Unternehmen

Ich zeige nur, was ich selbst gebaut und ausprobiert habe.



WAS EUCH ERWARTET

Die Evolution meines Vorgehens

2025
Vibe Coding

→

AUGUST
Software Factory

→

SEIT SEPTEMBER
Spec, Delta und Karte

An meinen eigenen Tools, neben der Arbeit. Was funktioniert, was nicht, und was ihr morgen ausprobieren könnt.
Work in Progress. Kein Tool zum Herunterladen, keine Studie.

DAS PROBLEM

Lesen wir jede Zeile, verschenken wir den Speed. Lesen wir sie nicht, verstehen wir unser System nicht mehr.

Addy Osmani nennt das **Comprehension Debt**.



Quelle: Addy Osmani, „Comprehension Debt“
https://addyosmani.com/blog/comprehension-debt/

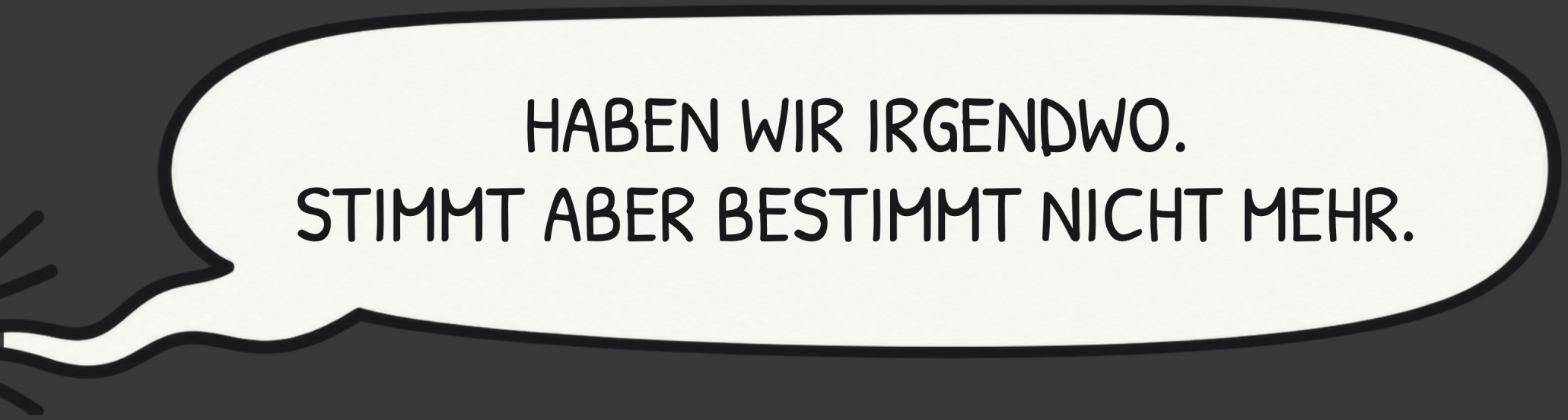
WAS PASSIERT, WENN NIEMAND MEHR LIEST

Lights off: vier Monate, dann abgeschaltet

Dex Horthy (HumanLayer) ließ ab Juli 2025 eine Factory laufen, deren Code kein Mensch las. Nach rund vier Monaten war Schluss: ein Fehler in Produktion, den das Modell nicht fand, und Wochen, um sich in ungelesenen Code einzuarbeiten.

Quelle: Dex Horthy bei The Pragmatic Engineer, 15.7.2026
https://newsletter.pragmaticengineer.com/p/context-engineering-with-dex-horthy

DIE ALTE ANTWORT



Aus dem Off: Kunden, immer wieder

Architektur-Doku entsteht am Anfang und driftet dann vom Code weg.

DIE WENDE

Aus dem Code generiert zeigt das Diagramm, was der Agent gebaut hat.



Ich will sehen, was ich bauen will, bevor er es baut.



Begriffe, wie ich sie benutze

Begriff	Bedeutung
Architektur	Die Systeme, ihre Verantwortung und die Verträge zwischen ihnen
Design	Wie ein einzelner Baustein innen gestaltet ist
Spec	Beschreibt das Ist-Verhalten, liegt im Repo
Delta-Spec	Beschreibt eine Änderung, liegt im Feature-Branch, wird beim Merge Teil der Spec
Soll-Manifest	Die geplante Struktur: Systeme und ihre Verträge. Daraus zeichnet sich die Karte

ERST DIE SPEC, DANN DER CODE

Spec im Repo, Änderung als Delta

Ist-Spec

liegt im Repo, beschreibt das heutige Verhalten

→

Delta-Spec

liegt im Feature-Branch, beschreibt die Änderung

→

Merge

das Delta wird Teil der Ist-Spec



Quelle: OpenSpec, Spec-Driven Development für KI-Coding-Assistenten
https://github.com/Fission-AI/OpenSpec

Mit Agenten wandert das Verstehen vom Code in die Spec.

Aber auch die Spec wächst schneller, als du lesen kannst.

Deshalb: Lies das Verhalten, und lass dir die Struktur zeigen.

NICHT ALLEIN

What-if, bevor gebaut wird

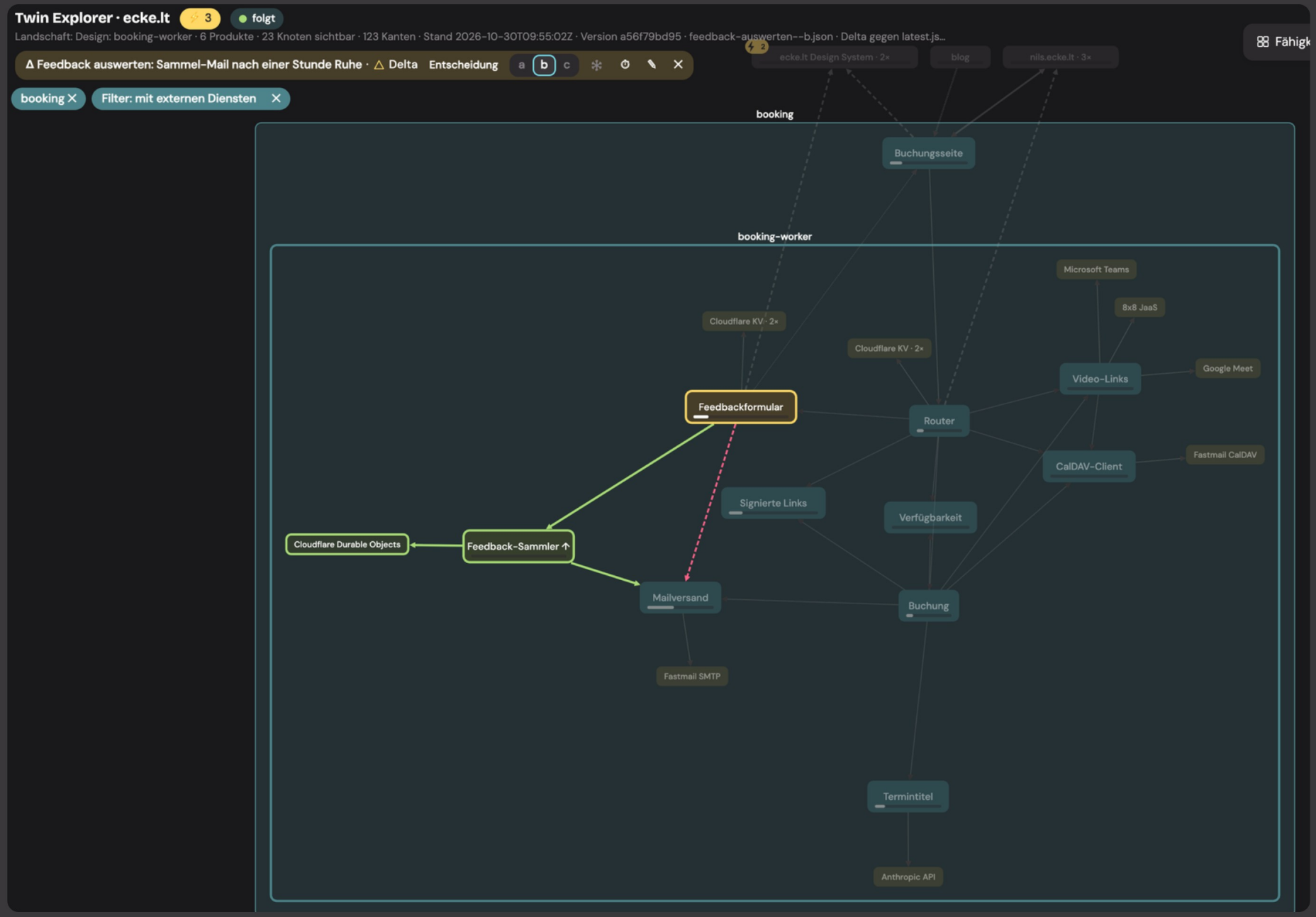
Uncle Bob zeigt ein UML-artiges Modell seines Systems, über das er mit seinem Agenten What-if-Spiele spielt. Der Agent schlägt eine Struktur vor, und erst wenn sie passt, wird daraus Code.

Quelle: Robert C. Martin (Uncle Bob) im Aviator-Podcast
https://www.youtube.com/watch?v=5kCISBJwoZo

DIE LANDKARTE

booking, mit einem Delta darauf

Gelb: geändert
Grün: neu
Rot gestrichelt: entfällt



Ein echtes Delta aus meinem Alltag

Feedback auswerten: eine Sammel-Mail eine Stunde nach dem letzten Feedback statt einer Mail pro Rückmeldung

1 · Idee

Im Chat: Ich will nicht jede Rückmeldung einzeln lesen.

2 · Delta-Spec

Eine Spec, drei Varianten im Wie:
A Cron
B Durable Object
C mit KI-Zusammenfassung

3 · Karte

In C hängt ein neuer Knoten am Feedbackformular: die Anthropic API.

4 · Aufgefallen

Moment: Dann gehen Rückmeldungen meiner Teilnehmenden an Anthropic.

5 · Zurück im Chat

Zwei neue Anforderungen: Datenschutzhinweis am Formular, Namen pseudonymisieren.

6 · Entscheidung: offen

Sicher ist nur: Wird es C, stehen die Datenschutz-Pflichten in der Spec statt im Hinterkopf.

Die Pflicht stand in keinem Ticket. Ich habe sie auf der Karte gesehen.

HAUPTTEIL

So entsteht ein Change auf der Karte

1 · Absicht

MENSCH

Ich lege einen Change an oder skizziere ihn.

2 · Übersetzen

AGENT

Schreibt das Soll-Manifest und die Delta-Spec in den Change-Branch.

3 · Zusammenführen

CODE

Branch auf main ziehen, Spec-Delta über die Ist-Spec legen.

4 · Verbinden

CODE

Kanten nur über gleiche Verträge. Was nicht passt, wird Drift.

5 · Vergleichen

CODE

Soll gegen Ist: neu, geändert, entfällt. Fester Layout-Seed.

6 · Umsetzen

AGENT

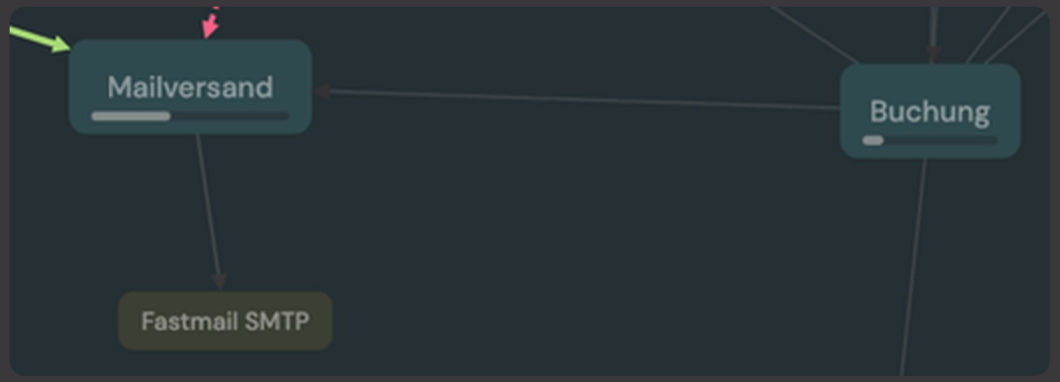
Baut Code und Tests. Beim Merge wird das Soll zum Ist.

Interpretiert wird nur in 2 und 6. Dazwischen ist alles Code: zwei Läufe, byteweise gleiches Ergebnis.

Mensch und Agent arbeiten auf demselben Modell.

Ich sehe es als Karte, der Agent liest es als Struktur.

ICH SEHE



DER AGENT LIEST

```
{
  "name": "Mailversand",
  "interfaces": [
    { "direction": "provides",
      "contractRef": "booking:worker-email@1" },
    { "direction": "consumes",
      "contractRef": "fastmail-smtp:smtp@1" }
  ]
}
```

WAS ICH GELERNT HABE

Hilft das gegen Comprehension Debt? — Nicht vollständig. 😖

Ich verstehe viel mehr. Lücken habe ich immer noch, aber ich merke sie jetzt und lasse sie mir vom Agenten erklären.

Diese Woche: Ich dachte, meine Karte entsteht aus der Spec. Erklärt bekommen: Sie entsteht aus dem Manifest.

Kollegentest ohne Erklärung: Die Ebenen hat er verstanden. Was Drift ist, nicht.

RANDNOTIZ: ABER DANN …

Spec ins leere Repo, Agent baut ohne Rückfragen

Sechs Läufe

mit der Landkarte selbst als Testobjekt

Specs aus Deltas

entstanden im Gespräch mit dem Agenten und durch eingemergte Deltas

Als Nächstes

das Buchungssystem, Specs dort aus dem Code erzeugt

Work in progress: Meine Specs enthielten noch Implementierungsdetails. Im Sinne von Dave Farley zählt eine Spec erst, wenn man sie beweisen kann. Mehr dazu in einem eigenen Talk.

Quelle: Dave Farley, Modern Software Engineering (YouTube, Oktober 2026)
https://www.youtube.com/watch?v=EkH_uoybj8k

ZUM MITNEHMEN

Vier Schritte, ohne mein Tool

1. Ist-Spec ins Repo
2. Änderungen nur als Delta-Spec. Reviewt wird das Delta, nicht der Code.
3. Die Struktur als Manifest daneben pflegen und daraus eine Übersicht zeichnen lassen. Mermaid reicht für den Anfang.
4. Die Spec mit einem Rebuild im leeren Repo testen.

Baut euch das Werkzeug, das ihr braucht. Meins ist nur ein Beispiel.

Wo das heute endet

Mein Labor

- Ein Ökosystem: Buchung, Blog, Profil
- Eine Person mit Agenten
- Kleine, eigene Systeme

Offen

- Viele Teams, viele Services
- Spec und Karte prüft niemand gegeneinander
- Halten Agenten Architekturvorgaben ein?

ZUM NACHLESEN

Die Folien zum Mitnehmen

Alle Folien als PDF, mit allen Quellen und Links.



ecke.lt/codetalks

Danke

Feedback geben oder 30 Minuten mit mir sprechen

Nils Eckelt · ecke.lt



feedback.ecke.lt/codetalks